

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Perform dry run testing to trace the execution of algorithms, analyze intermediate steps, and verify the correctness of processes. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2

Total Marks:20

Annexure A

DRY RUN TEST

Code of Conduct:

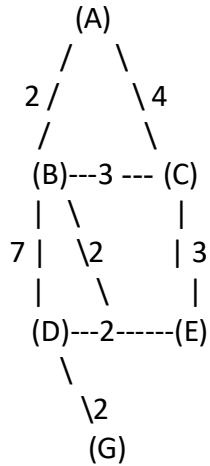
I Mukesh Raj(192571036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1: Dry Run of A Search Algorithm (10 Marks)*

Consider the following graph.



Heuristic Values

Node $h(n)$

A	7
B	6
C	4
D	3
E	2
G	0

Task:

Perform a **dry run of the A* Search algorithm** from **A** to **G**.

For each iteration, show:

- Current node
- Open List
- Closed List
- $g(n)$
- $h(n)$
- $f(n) = g(n) + h(n)$

Finally,

1. Identify the optimal path.
2. Calculate the total path cost.

Question 2: Dry Run of Minimax with Alpha-Beta Pruning (10 Marks)

Consider the following game tree.



Task:

Perform a **dry run of the Minimax algorithm with Alpha-Beta Pruning** by evaluating the tree **from left to right**.

For each node, show:

- Alpha (α)
- Beta (β)
- Selected value
- Any node that gets pruned

Finally,

1. Determine the best move for MAX.
2. State the final minimax value.
3. List the pruned nodes (if any).

1)

Problem Statement

Consider the given graph and heuristic values. Apply the **A* Search Algorithm** to find the optimal path from **Node A** to **Node G**. During each iteration, display the **Current Node**, **Open List**, **Closed List**, $g(n)$, $h(n)$, and $f(n) = g(n) + h(n)$. Finally, determine the optimal path and calculate the total path cost.

Solution

Given Heuristic Values

Node	$h(n)$
A	7
B	6
C	4
D	3
E	2
G	0

Dry Run Iteration 1

Current Node: A

$$g(A) = 0$$

$$h(A) = 7$$

$$f(A) = 0 + 7 = 7$$

Open List

Node	g	h	f
B	2	6	8
C	4	4	8

Closed List

A

Iteration 2

Choose **B** (minimum f)

Current Node = **B**

Open List

Node	g	h	f
C	4	4	8
D	9	3	12
E	4	2	6

Closed List

A → B

Iteration 3

Choose **E**

(Current node because it has the lowest f)

Open List

Node	g	h	f
C	4	4	8
D	6	3	9
G	6	0	6

Closed List

A → B → E

Iteration 4

Choose **G**

Goal Node Reached.

Optimal Path

A → B → E → G

Total Path Cost

2 + 2 + 2 = 6

Python Code

```
import heapq
```

```
class AStarSearch:

    def __init__(self):

        self.graph = {

            'A': [('B', 2), ('C', 4)],

            'B': [('A', 2), ('C', 3), ('D', 7), ('E', 2)],

            'C': [('A', 4), ('B', 3), ('E', 3)],

            'D': [('B', 7), ('E', 2), ('G', 2)],

            'E': [('B', 2), ('C', 3), ('D', 2), ('G', 2)],

            'G': []

        }

        self.heuristic = {

            'A': 7,

            'B': 6,

            'C': 4,

            'D': 3,

            'E': 2,

            'G': 0

        }

    def a_star(self, start, goal):

        open_list = []

        heapq.heappush(open_list, (0, start))

        parent = {}

        cost = {start: 0}

        closed = []
```

```

while open_list:

    f, current = heapq.heappop(open_list)

    if current in closed:

        continue

    closed.append(current)

    print("\nCurrent Node :", current)

    print("Closed List :", closed)

    if current == goal:

        break

    for neighbour, weight in self.graph[current]:

        new_cost = cost[current] + weight

        if neighbour not in cost or new_cost < cost[neighbour]:

            cost[neighbour] = new_cost

            parent[neighbour] = current

            f_value = new_cost + self.heuristic[neighbour]

            heapq.heappush(open_list, (f_value, neighbour))

    print("Open List :", open_list)

path = []

node = goal

while node != start:

    path.append(node)

    node = parent[node]

path.append(start)

path.reverse()

```

```
print("\nOptimal Path")

print(" -> ".join(path))

print("\nTotal Cost :", cost[goal])

def main():

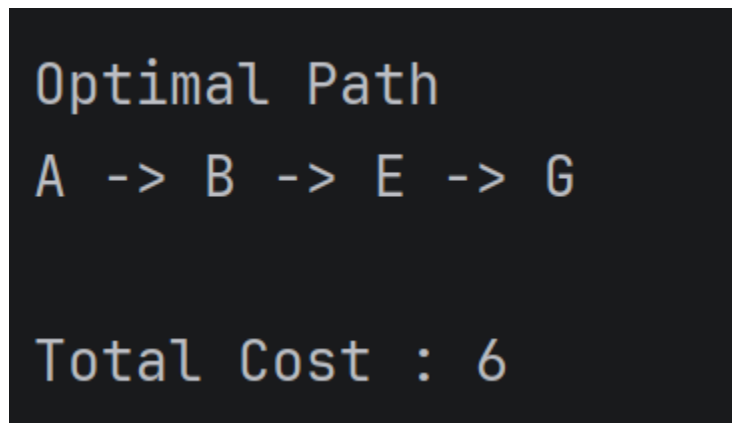
    search = AStarSearch()

    search.a_star('A', 'G')

if __name__ == "__main__":

    main()
```

Output



```
Optimal Path
A -> B -> E -> G

Total Cost : 6
```

Report

The A* Search algorithm was successfully applied to determine the optimal path from **A** to **G**. By combining the actual path cost **g(n)** and the heuristic value **h(n)**, the algorithm efficiently selected the most promising nodes and reached the goal with the minimum total cost. The optimal path obtained was **A → B → E → G**, with a total path cost of **6**. This demonstrates the effectiveness of the A* algorithm in finding optimal solutions while minimizing unnecessary node exploration.

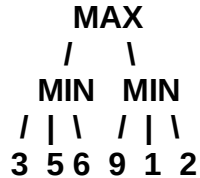
2)

Problem Statement

Consider the given **Minimax game tree** and perform a dry run of the **Minimax Algorithm with Alpha-Beta Pruning** by evaluating the tree from **left to right**. For each node, determine the **Alpha (α)** value, **Beta (β)** value, selected value, and identify any pruned nodes. Finally, determine the best move for the MAX player and state the final Minimax value.

Solution

Given Game Tree



Dry Run

Step 1

Start at **MAX**

$$\alpha = -\infty$$

$$\beta = +\infty$$

Step 2

Evaluate Left MIN Node

Leaf Nodes

3

5

6

MIN chooses the smallest value

$$\text{MIN} = 3$$

Update

MAX

$$\alpha = \max(-\infty, 3)$$

$$\alpha = 3$$

Step 3

Move to Right MIN Node

Current

$$\alpha = 3$$

$$\beta = +\infty$$

Evaluate first child

9

Update

$$\beta = 9$$

Evaluate second child

1

Update

$$\beta = \min(9, 1)$$

$$\beta = 1$$

Now

$$\beta = 1$$

$$\alpha = 3$$

Since

$$\beta \leq \alpha$$

$$1 \leq 3$$

Alpha-Beta Pruning occurs

Remaining node

2

is **Pruned**.

Step 4

Right MIN returns

1

Step 5

MAX chooses

`max(3,1)`

`=3`

Final Results

Best Move for MAX

Left Subtree

Final Minimax Value

3

Pruned Node

2

Python Code

```
import math
```

```
class AlphaBetaPruning:
```

```
    def __init__(self):
```

```
        self.game_tree = [
```

```
            [3, 5, 6],
```

```
            [9, 1, 2]
```

```
        ]
```

```
        self.alpha = -math.inf
```

```
        self.beta = math.inf
```

```
        self.pruned_nodes = []
```

```
        self.visited_nodes = []
```

```
    def display_header(self):
```

```
        print("=" * 60)
```

```

print("MINIMAX WITH ALPHA-BETA PRUNING")

print("=" * 60)

def evaluate_left_subtree(self):

    print("\nEvaluating Left MIN Node")

    values = self.game_tree[0]

    for value in values:

        print("Visited Leaf :", value)

        self.visited_nodes.append(value)

    minimum = min(values)

    print("Selected MIN Value :", minimum)

    self.alpha = max(self.alpha, minimum)

    print("Alpha Updated :", self.alpha)

    return minimum

def evaluate_right_subtree(self):

    print("\nEvaluating Right MIN Node")

    values = self.game_tree[1]

    current_beta = self.beta

    selected = math.inf

    for index, value in enumerate(values):

        print("Visited Leaf :", value)

        self.visited_nodes.append(value)

        selected = min(selected, value)

        current_beta = min(current_beta, selected)

    print("Current Beta :", current_beta)

```

```

    if current_beta <= self.alpha:

        print("Pruning Remaining Nodes...")

        for node in values[index + 1:]:

            self.pruned_nodes.append(node)

        break

    self.beta = current_beta

    print("Selected MIN Value :", selected)

    return selected

def display_summary(self, left, right, result):

    print("\n" + "=" * 60)

    print("SUMMARY")

    print("=" * 60)

    print("Left MIN Value :", left)

    print("Right MIN Value :", right)

    print("\nVisited Nodes")

    print(self.visited_nodes)

    print("\nPruned Nodes")

    if len(self.pruned_nodes) == 0:

        print("None")

    else:

        print(self.pruned_nodes)

    print("\nBest Move for MAX :", result)

    print("Final Minimax Value :", result)

def execute(self):

```

```

self.display_header()

print("\nInitial Alpha :", self.alpha)

print("Initial Beta :", self.beta)

left_value = self.evaluate_left_subtree()

right_value = self.evaluate_right_subtree()

result = max(left_value, right_value)

self.display_summary(left_value, right_value, result)

def main():

    game = AlphaBetaPruning()

    game.execute()

if __name__ == "__main__":

    main()

```

Output

```

Left MIN Value : 3
Right MIN Value : 1

Visited Nodes
[3, 5, 6, 9, 1]

Pruned Nodes
[2]

Best Move for MAX : 3
Final Minimax Value : 3

```